Object Oriented Programming Final Report

B99611001 李弘宇 B99902055 陳勁宇 B99902078 王立

Division of the responsibilities

李弘宇	戰鬥畫面
陳勁宇	地圖生成、主角移動、劇情串聯、NPC
王　立	選單、道具、存檔、特效、音效、遊戲進入畫面

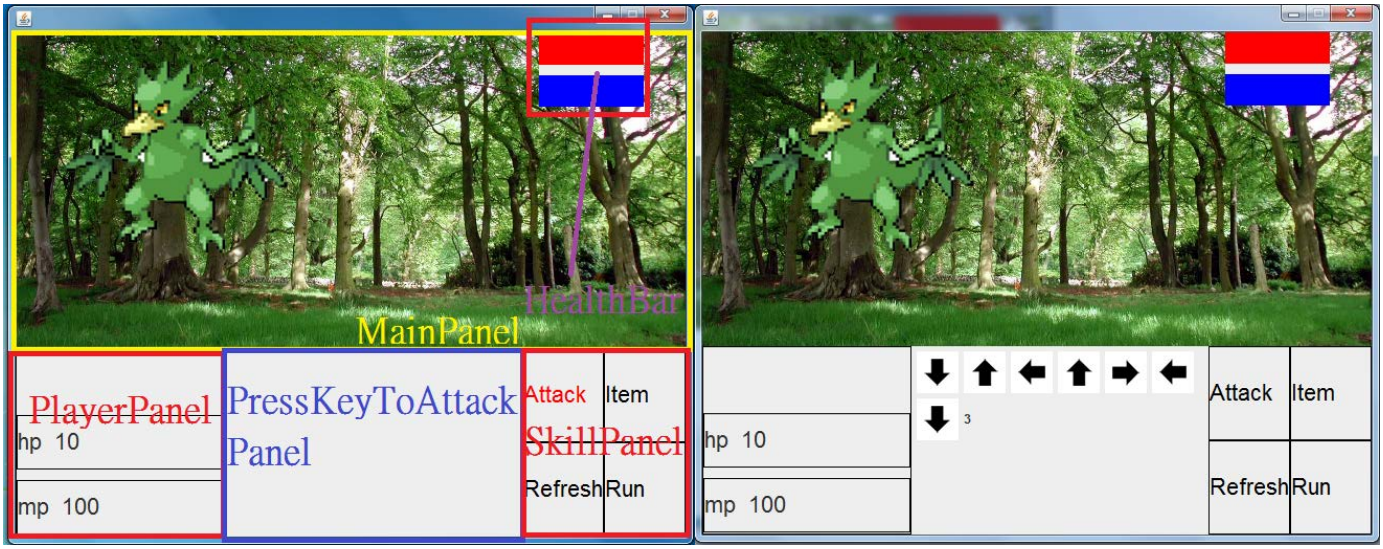
The relations between the classes that you design

Character, Monster, Player:

Character 是個繼承 JPanel 的抽象 class，而主要使用到的人物是 Monster 和 Player，通過這樣的設計，我的其他 Panel 可以以 Character 當做 argument，呼叫他本身就有的 public method 或等待其他 subclass override 的 abstract method。這讓我設計 Skill 等不同 class 時有很大的方便。然而此 abstract class 是以會進入戰鬥畫面的人物為主（有可以操作 HP, MP 的函式），因此 NPC—只會根據劇情述說固定對話、站著不動的人物—在設計上並沒有繼承。這似乎不符合大家的直覺，NPC 也應該算是人物之一。

FightPanel, MainPanel, PlayerPanel, PressKeyToAttackPanel, SkillPanel, HealthBar:

在 class FightPanel 中寫著的是代表戰鬥畫面的所有事情，它內部所有的 Panel 都被我拆開來一一設計，起因於課堂上說過的"has a"的概念，FightPanel 中有（has a）代表主畫面的 MainPanel（印出戰鬥的背景和怪物的圖片）、顯示出 Player 狀態的 PlayerPanel（顯示其 HP, MP）、中間有著必須按對上下左右鍵後才能出招的 PressKeyToAttackPanel（當 Player 選定招式後，必須按下隨機出現在這個 Panel 上面的上下左右按鍵），才能成功發出招式、中間）、顯示出技能的 SkillPanel（在右下角的技能選單）、當選擇道具時會跳出 item 的 Panel，必須按 ESC 才能跳出、最後是顯示怪物的 HP/MP 值的 HealthBar class。附上兩個戰鬥畫面應該可以比較清楚的了解這段文字的敘述。



FightPanelListener, PressKeyToAttackPanelListener, SelectSkill, SelectAttackSkill,

最後則是要處理各式 swing 裡面各式事件的 actionListener 以及操作選單的 extends KeyAdapter 的東西，這些 class 都是負責處理 swing 的事件，以及負責 Focus 的轉移（requestFocusWindows），例如 FightPanelListener、SelectSkill、SelectAttackSkill、FightPanelListener 等等的，這些狀態間的轉移都要仔細的考慮清楚，也因此花了我不少時間。

Background: 繼承自 JPanel，原主要目的在顯示背景地圖及背景音樂的加入。後來因為各個 class 之間協調所需，漸漸變成 Player、Map、Dialog 之間的溝通管道。主要的工作有：轉換地圖時的轉換音效、NPC 設置、道具設置，及檢查某塊地區是否可走、是否有 NPC、是否有道具等功能。

Map: 轉換一張地圖時，Map 的工作就是把地圖資訊，也就是在 data/mapdata 資料夾裡對應的檔案資訊給 load 出來。也就是說，Map 代表的就是玩家現在所在的地圖。Background 如果需要地圖資訊時，會跟 Map 要。其中包括紀錄該地圖通往別張地圖的通道、該地圖的背景音樂、道具、NPC 位置等。

Player: 繼承自 Character，而 Character 又繼承自 JPanel，所以它是一個 JPanel 的 extension。主要在記錄玩家的一些行為，包括行走、HP、MP、道具、金錢，和能力值。當玩家在行走時，Player 是站在第一線和 KeyListener 溝通，以達到讓主角上下左右行走的目的。但由於最後的缺失，導致 Player 裡包含了許多和玩家性質無關的 method。例如商店內店員的行為，和一些會觸發 NPC 行為的 method。另外，Player 還扮演著和 FightPanel 互動的角色，戰鬥事件都是由 Player 所觸發。

NPC: 繼承自 JPanel，每個 instance 代表的一個 NPC。主要的工作是控制 NPC 的對話內容。NPC 會和 Background 互動，每次玩家進入一個新的地圖，地圖內的 NPC 會一一被 Background 所創建。

Status: 紀錄遊戲進行到哪個階段。例如劇情發展到什麼階段，每個 NPC 的對話的 stage。基本上 Status 不會記錄詳細的劇情內容(例如 dialog 內容)，而是只會紀錄 stage。例如，它可以會記錄 5 號 NPC 目前會說 index=5 的 dialog，NPC 再根據這個 5 去查檔案，得到詳細的 dialog 內容。

BGM: 背景音樂播放。

Dialog: 繼承自 JPanel，負責呈現對話。在 init 完成後使用 setContent 將會自動呈現對話，因為是使用 Timer 去排程，可以使用 setContent 確認是否完成對話來達成和 player 的同步。

FadingPanel: 繼承自 JPanel，負責表現淡入、淡出效果(遊戲中沒有使用到淡入)。在 init 完之後使用 fadeIn 或 fadeOut 會自動呈現淡入、淡出。因為是使用 Timer 去排程，可以使用 isRunning 確認是否完全淡出或淡入來達成和 player 的同步。淡出、淡入的方法則是擷取某個 JPanel 的目前顯示狀態(畫到一個 Graphics object 上)，然後對其進行透明度的調整。

Item: 一個 abstract class，用來表示所有道具的 super class。

ItemList: 繼承自 JPanel，負責呈現道具欄，有三種 mode，分別是戰鬥中用(選擇後立刻結束)、player 的道具資料的顯示(可以選擇並使用該道具)、商店用(可以買選擇的道具)。

Option: 繼承自 JLabel，負責顯示按下 esc 鍵後出現的選單的單一選項。

OptionActor: 一個 interface，負責用來註冊偵測到事件後要觸發的 method。

PlayerOption: 實作 OptionActor 介面，也就是註冊了 player 的 method 的 action performer。

OptionList: 繼承自 JPanel，負責顯示按下 esc 鍵後出現的選單。

PlayerStats: 繼承自 JPanel，顯示 player 的狀態諸如 HP、M 等等。

SingleItem: 繼承自 JPanel，負責顯示地圖上的道具(寶貝球圖示)，當被撿到(pick method)，會根據其 private member: type 來決定要讓撿到的 player 得到哪種道具。

StartMenu: 繼承自 JPanel，遊戲的初始畫面，會另外開一個畫面覆蓋在 main frame 上，可以透過 choicemethod 來達成 main thread 和其之間的同步，並決定是要開新遊戲還是讀取存檔。

The advantages of your design

李弘宇：

在寫了許許多多的 Timer 和 actionListener 後，我認為有兩個 class 很明顯地可以應用到 Design Pattern 書中的 Observer Pattern，也就是利用 java.util.Observer 這個 interface 和 java.util.Observable 這個 class 所組合而成的觀察者模式。有個 class 為 Sensor，這個 class 繼承了 Observable，他可以傳遞一些訊息給 Observer，這個 pattern 剛好可以用在 Health bar 與 PlayerPanel 上面，因為他們的值隨時都必須根據 Player 和 Monster 這兩個對戰角色而變，也就是他們都是參考同一份資料的，當 Player 以及 Monster 的數值改變時，其內部的 Sensor instance 便會 notifyObservers()，通知 implement Observer 的 HealthBar 和 PlayerPanel 做出改變，而且同時可以判斷戰鬥是否已經結束也（就是 HP=0）。

另外，我還面臨到老師上課所把玩的 swing 的小程式所提到的設計方式，ActionListener 到底是要自己兼任還是要另外寫個專任的 class implements ActionListener 呢？我認為分開來寫可以很明顯的藉由物件的名稱看出這是一個 Listener，專門用在其他 class 上面。

另外，戰鬥畫面每個 Panel 都分開來寫可以使得 FightPanel 要做的事情分的清楚，只要負責將每個 Panel 置入自己的 Panel 中，而且每個 Panel 都可以使用自己應該使用的 Layout 方法，而不是根據絕對位置畫出（無 Layout 時)，各自負責自己所需要的資訊使得架構變得清楚。

陳勁宇：

在地圖、主角、NPC 和劇情方面，我們是將資料寫在檔案裡，程式在執行的時候會進行讀檔。其中的優點是，如果其中的內容須要被修改，程式不須重新被編譯。以地圖為例，我們是用一張背景加上一張底圖來實作。背景就是玩家可見的部分，底圖在各個點有不同的灰階值，而我們利用這些灰階值來判斷這個點是否可走、是否會遇到怪獸等狀況。如果我們要對一張圖進行更動，例如更改其中房屋的位置，我們只要單純更換圖檔即可，不須對程式進行編譯。類似的情形適用於劇情的交代等細節的部份，如果要進行劇情的改編，只要對 data 資料夾下的檔案進行對應的修改即可。

即使每個 NPC 都具有全然不同的功能，以上的寫法讓我們得以以一個 class 來涵蓋多個 NPC。每個 NPC 的 instance 都會先進行讀檔，得知自己的任務之後，才出現在遊戲之中。這樣的方法讓我們減少程式數量，以及編譯的次數。

需要的資料會在需要的時刻才被 load 出來，而非程式的一開始就全數 load 完畢，程式在初始階段會因此省時許多。基本上，當我們走入其中一個地圖時，有關該圖的資料才會被 load 出。雖然時常需要做 loading，但因為每次的資料量都不大，所以可算是相當有效率。

趣味性方面，劇情的部分嘗試著讓難易度適中。整個故事的事件約為 15 個，玩家必須在過程中找到關鍵人物談話或戰鬥，才可以開啟下一個事件。關卡其中有一兩個稍具難度，但透過存檔的機制讓玩家在失敗時不用大老遠的從頭開始。最後讓劇情結合時事，大體上算是難度適中，又不失趣味性。

王立：
這個 project 裡我負責的大部分都是小元件的撰寫，諸如淡出、起始選單、記錄、顯示選項，我的 design 的希望是在不影響大架構的情況下能簡單地擴充遊戲的一些功能或特效，原先的預期是只要呼叫像是 init、start 等 method 後，我設計的小 class 便會自動完成任務以達到擴充的效果。在初期的開發中，我認為我和組員之間的整合尚算順利，我也盡量遵守這學期所學去封裝一個 class，讓我的組員在使用我的 class 上並不會感到太大的不便。

The disadvantages of your design

李弘宇：
設計了戰鬥畫面，我發現我構造這些 class 的方向是由下而上的，也就是 FightPanel 幾乎無法獨立於其他 Panel 而生，而且彼此的相依性、耦合性太高，少了 FightPanelListener 許多事情都無法達成，包括回合的轉換、Focus 的轉移等等，也就導致 FightPanelListener 這個 ActionListener 所做的事情非常複雜，十分的難以維護。幾乎每加入一個新的 State 就要重新更動這個 class。幾次下來，我幾乎可以確定這個class絕對是我優先要改變架構的class。這件事情是我從一本有關Refactoring的書上所看來的。當然，由於 interface 等架構一開始設計不良，導致後面的設計常常需要修改 interface，以及同一個 class 包含了太多不屬於自己的資料，擁有許多 method 處理的都不是自己 class instance 的資料，這些缺點都因為時間的壓力而逐漸浮現，十分明顯。當然還有一個 method 做了太多事情（冗長 method 可以拆開成為小的 method）、一個地方出現了許多的 switch 和 if-else（例如 FightPanelListener 之中判斷狀態轉移以及 KeyBoard Focus 轉換的地方等等），這些東西都是值得我逐一改進的地方，也是我的缺點。
最後，FightPanel 利用了諸多的 get method 直接對其他 class 之中的 instance 進行操作，這個違反了 OOP 的 encapsulation 原則，這個部分也是值得我去改進的地方。

陳勁宇：
Player 在程式中佔了很重要的一部分。一開始我們在寫這個 class 的時候，嘗試讓它控制著有關玩家的一些參數，包括位置、體力等。但由於最後在急忙之間，沒有顧慮到整個架構性，所以 Player 到最後包括了許多不相干的內容，例如商店、選單都包含於其內。
Background 也有類似的情形，一開始是想要拿來加入背景的 panel，最後卻淪為玩家和 NPC 或道具互動的 class。這樣的寫法讓程式很難被再被 extend 和 maintain。
趣味性方面，由於我們一開始低估地圖的大小，我們最後發現要走完全程需要堅強的意志力。由於 NPC 和地圖之間的掌控不佳，可能要走很長的距離才會遇到一次 NPC，或者走兩到三個地圖的距離才會到達下個任務點。對於這類小型的遊戲，會讓玩家玩久了之後失去興趣。

王立：
嚴格來說，這次的 project 是我第一次需要處理 swing 元件之間的互動，雖然在前幾次的作業中都用到了 swing 的圖形效果，但對其了解並不深入。在這次 project 中知道了 swing 元件的處理都是交給 event dispatching thread 來處理，是 single-threaded 的方式，因為這樣的疏忽而造成在一個 JPanel 中 sleep，卻讓整個遊戲停擺的狀況。因此後來得把架構從 single-threaded 改成 multi-threaded(新增一個 thread 或是使用 invokeLater 的方式去處理和時間有相關的事件)。因為事前對 swing 的不夠了解而造成之後把一些 class 的架構寫得雜亂無章，即使 method 和用法基本上仍然不變，但為了達到 thread

間的同步，也必須修改主要架構，總的來說，雖然完成了 multi-thread 的同步，但程式碼已成了一盤炒麵。

How to play your RPG

利用上下左右鍵控制文物在地圖中的方向。長按 X 是快跑。

在地圖中對 NPC、道具按下 Z 可以進行對話，或拿起道具。

在選單內用上下移動，按下 ENTER 可以選取。

在地圖上按下 ESC 可以叫出選單或跳出選單。